PAIRWISE LAMBDARANK

Learn preferences. Spend effort where ranking quality moves.

Our implementation trains on ordered document pairs, weights every correction by its top-K NDCG impact, and maintains a latent document ranking.

1 PAIR EVIDENCE

Score an ordered comparison

One tree row represents (i, j):

$$[x_i \mid x_j \mid \text{optional } (x_i - x_j)] \rightarrow f(i,j)$$

Direct margin

$$d_{ij} = \text{mean}\{f(i,j), -f(j,i)\}$$

Optional indirect evidence through document k

a_{ij} : $f(i,k) + f(k,j)$ and signed variants

Blended preference margin

$$m_{ij} = \frac{w_d d_{ij} + w_a a_{ij}}{w_d + w_a}$$

$w_d = 1 - \alpha,\ w_a = \alpha$ when that evidence exists

2 NDCG-AWARE LAMBDA

Price each mistake by its ranking impact

For unequal labels, with at least one item currently in top K:

Swap value

$$\Delta\text{NDCG}_{ij} = \frac{|G(y_i) - G(y_j)|\ |D(r_i) - D(r_j)|}{\text{IDCG@K}}$$

$G(y) = 2^y - 1$ by default | $D(r) = 1/\log_2(r+2)$

Logistic pressure

$$q_{ij} = \frac{1}{1 + e^{\sigma m_{ij}}}$$

mass on the wrong order

LambdaRank pseudo-gradient and curvature

$$\lambda_{ij} = -\sigma q_{ij} \Delta\text{NDCG}_{ij}$$

$$h_{ij} = \sigma^2 q_{ij}(1 - q_{ij})\,\Delta\text{NDCG}_{ij}$$

3 RANKING STATE

Reconstruct the list

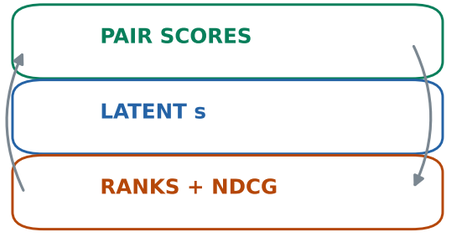
Maintain latent scores s_i that determine current ranks.

Match preference probabilities

$$P_{pair} = \text{sigmoid}(\sigma m_{ij})$$

$$P_{rank} = \text{sigmoid}(\sigma(s_i - s_j))$$

A damped Newton-style step aligns them; every score change is clipped to +/-0.3.



IN CODE | top-K truncation | optional LambdaRank normalization | item-weight product | log discount / hard preference / L2 controls

MANAGER TAKEAWAY

Pairwise learning tells us who should win; NDCG weighting tells us which wins matter most.

Implementation anchors: src/objective/rank_objective.hpp + .cpp; src/io/metadata.cpp; src/io/dataset.cpp